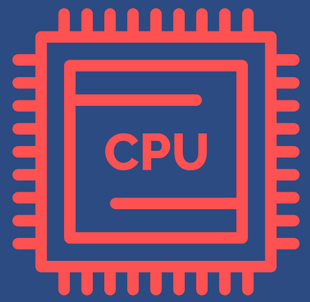
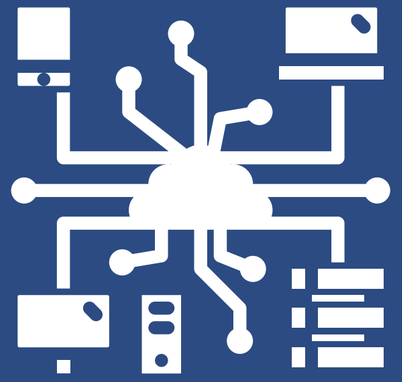
Jorge Durán Chinchilla: C02679





Servidor de simulaciones FDTD concurrente



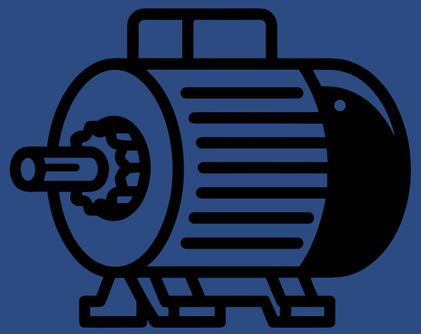
Parte A. Motor FDTD y Barrera Reutilizable.

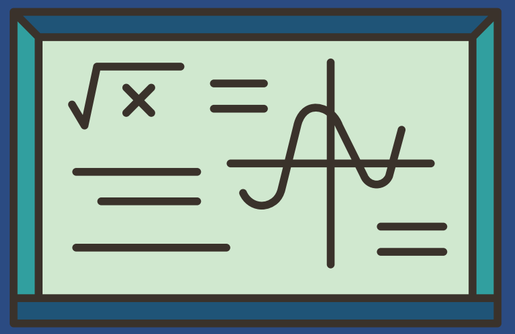
Función del Motor FDTD

- Resuelve Maxwell en malla nx×ny con 3 campos: Ez, Hx, Hy

- Avanza pasos de tiempo con T hilos

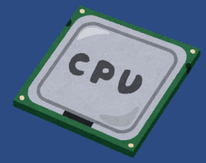
- Se comunica con el resto por callbacks

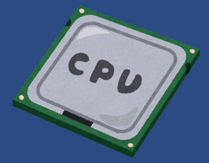


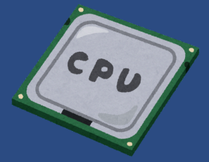


Paralelización: Reparto + Barrera

- Reparto de filas [nx * i/T, nx * (i+1)/T]
- H → BARRERA 1 → E + fuente → BARRERA 2 → callbacks








```
    for (paso = 1; paso <= s->pasos; paso++) {
        update_h(s, arg->row_begin, arg->row_end);
        LAP(arg->t_compute);
#ifndef SIN_BARRERA
        barrier_wait(&s->barrier);
#endif
        LAP(arg->t_barrier);
        update_e(s, arg->row_begin, arg->row_end, paso - 1); /* q = 0..pasos-1 */
        LAP(arg->t_compute);
#ifndef SIN_BARRERA
        barrier_wait(&s->barrier);
```

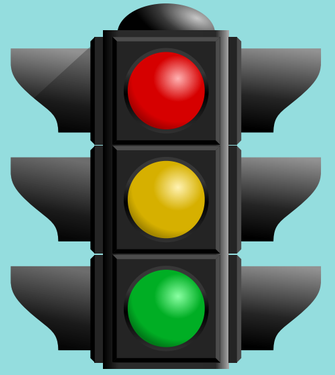
Barrera reutilizable

```c
void barrier_wait(barrier_t *b)
{
    int my_phase;
    int i;
    int is_last = 0;

    if (b == NULL || b->total <= 1) {//si la barrera es nula o el número total de hilos es me
        return;
    }

    (void)pthread_mutex_lock(&b->mutex);//bloqueo con mutex para proteger el acceso a la barr
    my_phase = b->phase;
    b->waiting++;
    if (b->waiting == b->total) {//si todos los hilos han llegado a la barrera, se desbloquea
        b->waiting = 0;
        b->phase = 1 - b->phase;//cambio de fase para la siguiente barrera
        is_last = 1;
    }
    (void)pthread_mutex_unlock(&b->mutex);

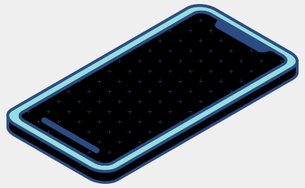
    if (is_last) {//si es el último hilo en llegar a la barrera, desbloquea a todos los demás h
        for (i = 0; i < b->total - 1; i++) {
            (void)sem_post(&b->turnstile[my_phase]);
        }
    } else {
        (void)sem_wait_nointr(&b->turnstile[my_phase]);//si no es el último hilo,debe esperar a
    }
}
```

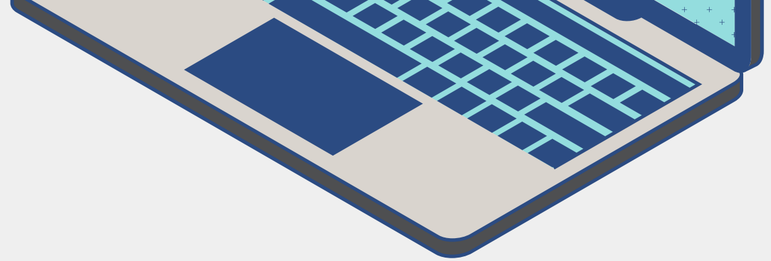


- Mutex, un contador , dos semáforos.

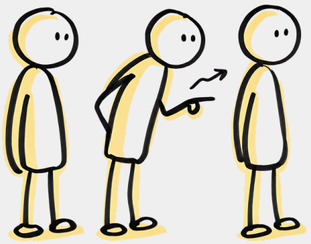
- Con dos semáforos alternados, cada ronda usa el suyo







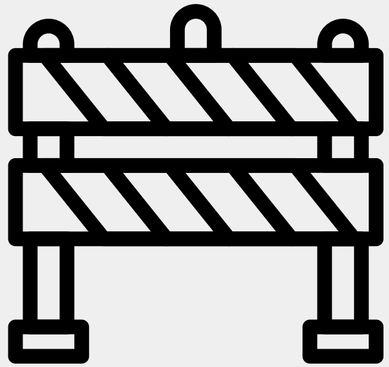
Determinismo: T = 1, 2, 3, 4, 8
→ idéntico bit a bit
(memcmp)



¿Por qué? : Mismas operaciones por celda - Sin reducciones paralelas - Suma final secuencial

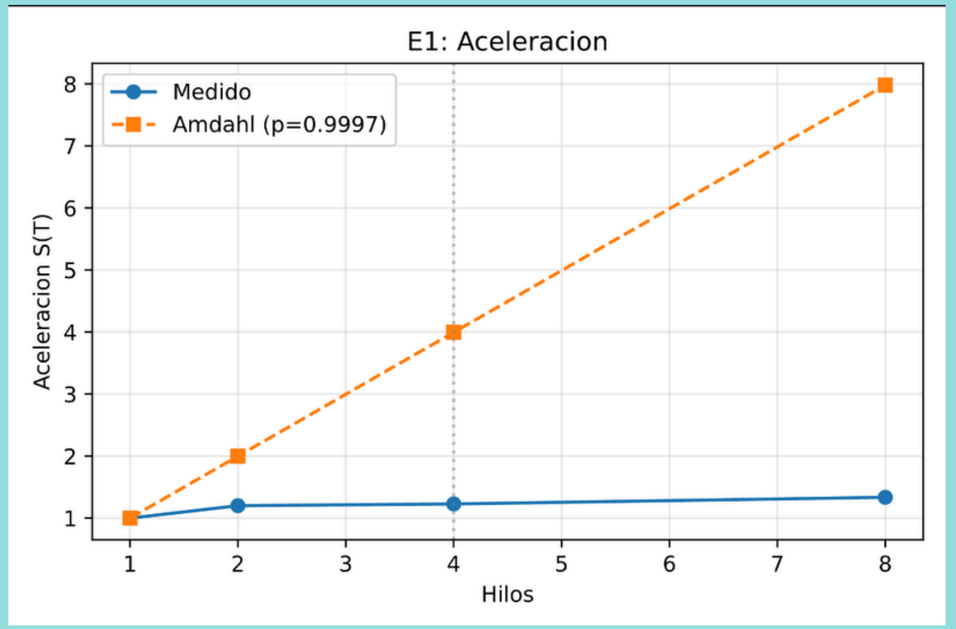
Determinismo y - DSIN_BARRERA

-DSIN_BARRERA: 20/20
corridas detectaron diferencias



Diapositiva 6 — Resultados: Amdahl y Karp-Flatt

T	Mediana	Rango	S(T)	E(T)	e(T)
1	21.5706	20.4140 – 24.3467	1.000	1.000	—
2	17.9586	16.8236 – 19.5935	1.201	0.601	0.666
4	17.5525	14.4874 – 19.4869	1.229	0.307	0.752
8	16.1455	14.5187 – 19.2256	1.336	0.167	0.712

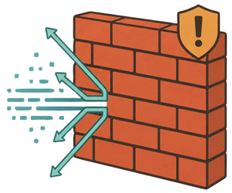


Resumen

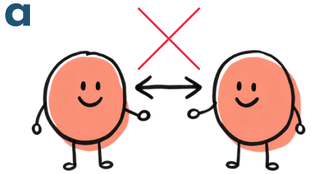
- Motor FDTD paralelo correcto: determinismo bit a bit



- Barrera reutilizable: 2 semáforos alternados



Sin barreras: 20/20 detecta la carrera

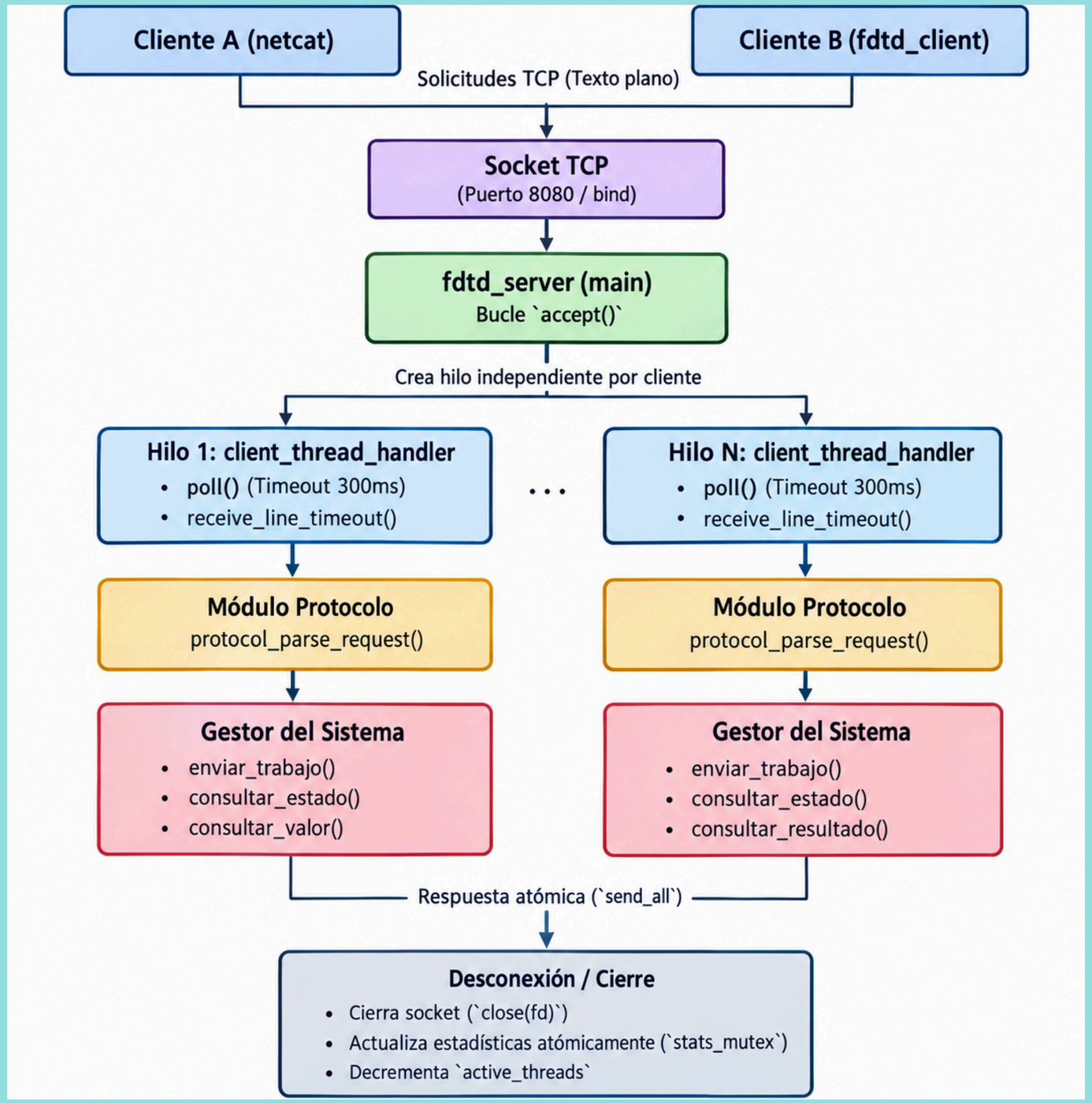


Speedup real 3.99× vs 1.229× teórico



- Limitado por sincronización y memoria

Arquitectura del Servidor Concurrente (Thread-per-Client)



Módulo de Protocolo y Validación Estricta (protocolo.c)

Comandos Soportados

- SUBMIT: Envío de parámetros.
- STATUS: Consulta del estado actual de un trabajo.
- PEEK: Consulta de un valor puntual en la matriz.
- RESULT: Obtención del resultado final y archivo de salida.
- SHUTDOWN: Apagado ordenado del servidor.

Seguridad y Validaciones (Parser)

- Detección de errores sintácticos y léxicos.
- Validación de límites físicos de la malla.
- Restricción de hilos respecto al tamaño.
- Control de rangos de la fuente para evitar desbordamientos.

Apagado Ordenado y Resultados

- Lectura segura con poll() (timeout de 300 ms).
- Función send_all contra fragmentación de red.
- Manejo de señales (SIGINT / Ctrl + C) y nanosleep.

- Casos de prueba evaluados: 44 escenarios automatizados. (100% de éxito).
- Cobertura: Parámetros límite, comandos inválidos, tipos de datos erróneos y formato exacto de respuestas.

Despachador y datos compartidos

Separación de responsabilidades y acceso seguro a la matriz Ez

dispatcher.c

- Toma los trabajos de la cola
- Ejecuta FDTD
- Actualiza el progreso

shared_state.c

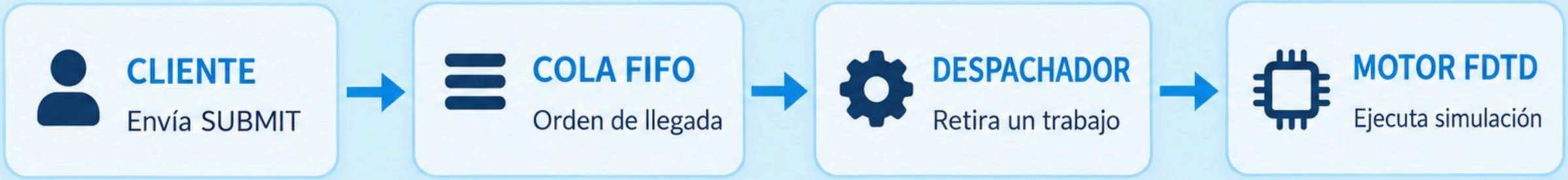
- Publica la matriz Ez
- Permite consultas PEEK
- Conserva el estado actual

rwlock.c

Varios lectores simultáneos / Un escritor exclusivo

Gestión de trabajos

De la solicitud del cliente a la ejecución de la simulación



queue.c

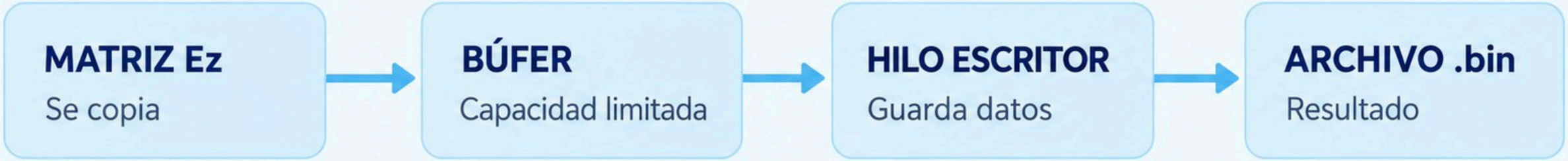
- Capacidad limitada
- Si se llena: **BUSY**

job_manager.c

- Registra progreso y estado
- **QUEUED** **RUNNING** **DONE** **ERROR**

El reto: snapshot.c

Coordinar la creación de instantáneas con su escritura en disco



Tres operaciones clave

```
sem_wait(&espacios);
```

Espera espacio libre

```
memcpy(copia, ez, total * sizeof(double));
```

Copia independiente de Ez

```
sem_post(&disponibles);
```

Avisa al hilo escritor

Pruebas y resultados

Resultados registrados en la documentación del proyecto

Prueba	Caso	Local
Cola FIFO	Orden, capacidad, posición y cierre	**OK**
Gestor de trabajos	Registro, estados, progreso y resultado	**OK**
Lectores-escritores	Lecturas simultáneas y escritura exclusiva	**OK**
Instantáneas	Archivo binario, encabezado y valores Ez	**OK**

Cuatro pruebas de los módulos concurrentes con resultado local OK.